Seaborn Visualization Cheatsheet

Assume:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

df = sns.load_dataset("tips")
```

The `tips` dataset contains columns such as:

```
total_bill    tip    sex    smoker    day    time    size
```

---

1. Line Chart — `sns.lineplot()`

What

Shows **trends over an ordered variable**, especially time.

When

Use when you want to answer:

> "How does X change as Y progresses?"

```
sns.lineplot(
    data=df,
    x="day",
    y="total_bill"
)

plt.show()
```

With aggregation

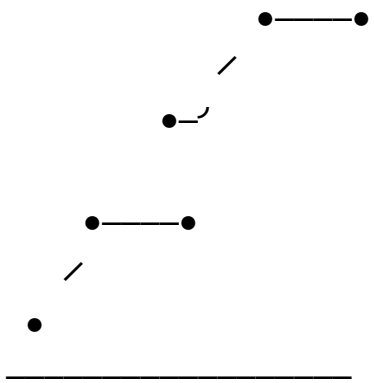
Seaborn automatically estimates an aggregate when multiple observations share the same x-value.

```
sns.lineplot(
    data=df,
    x="day",
    y="total_bill"
)
```

Multiple lines

```
sns.lineplot(
    data=df,
    x="day",
    y="total_bill",
    hue="sex"
)
```

Result conceptually:



Useful parameters

```
sns.lineplot(
    data=df,
    x="day",
    y="total_bill",
    hue="sex",
    style="smoker",
    markers=True,
    dashes=False,
    errorbar=None
)
```

Remember:

```
x       → ordered variable
y       → numerical variable
hue     → color/group
style   → line style/group
markers → show points
```

---

2. Bar Chart — `sns.barplot()`

What

Compares a **numerical statistic across categories**.

```
sns.barplot(
    data=df,
    x="day",
    y="total_bill"
)

plt.show()
```

By default, you're generally looking at an **estimate of the mean** for each category, with uncertainty shown.

Grouped bars

```
sns.barplot(
    data=df,
    x="day",
    y="total_bill",
    hue="sex"
)
```

Conceptually:

```
Total Bill

Mon  ██████
     ████

Tue  ████████
```



Wed 



Change estimator

For example, median:

```
sns.barplot(
    data=df,
    x="day",
    y="total_bill",
    estimator="median"
)
```

Or sum:

```
sns.barplot(
    data=df,
    x="day",
    y="total_bill",
    estimator="sum",
    errorbar=None
)
```

Horizontal bar

Swap X and Y:

```
sns.barplot(
    data=df,
    x="total_bill",
    y="day"
)
```

Important distinction

Don't confuse:

```
sns.barplot()
```

with:

```
sns.countplot()
```

`barplot`:

> category → numerical statistic

`countplot`:

> category → number of observations

---

3. Scatter Plot — `sns.scatterplot()`

What

Shows the relationship between **two numerical variables**.

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip"
)

plt.show()
```

Each dot = one observation.

```
Tip
 ^
 |              •
 |           •
 |        •
 |     •
 |  •
 └──────────────────> Total Bill
```

Add groups

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
```

```
    hue="sex"
)
```

Change marker style by group

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
    hue="sex",
    style="smoker"
)
```

Size based on another variable

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
    hue="sex",
    size="size"
)
```

Useful parameters:

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
    hue="sex",
    style="smoker",
    size="size",
    alpha=0.7
)
```

---

4. Regression Plot — `sns.regplot()`

What

A scatter plot **plus a fitted regression line**.

```
sns.regplot(
    data=df,
    x="total_bill",
    y="tip"
)

plt.show()
```

Conceptually:

```
Tip
 ^
 |                  •
 |               •
 |            •
 |         •
 |      ⁄
 |    ⁄
 |  ⁄
 └──────────────────> Total Bill
```

It helps answer:

> "Is there a linear relationship between X and Y?"

`regplot()` with options

```
sns.regplot(
    data=df,
    x="total_bill",
    y="tip",
    scatter_kws={"alpha": 0.5},
    line_kws={"color": "red"}
)
```

Polynomial regression

You can fit a higher-order polynomial:

```
sns.regplot(
    data=df,
```

```
    x="total_bill",
    y="tip",
    order=2
)
```

`order=2`:

quadratic curve

`order=3`:

cubic curve

Be careful with high-order regression because it can overfit.

---

5. `regplot()` vs `scatterplot()`

Scatter

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip"
)
```

Question:

> What relationship/pattern exists?

Regression

```
sns.regplot(
    data=df,
    x="total_bill",
    y="tip"
)
```

Question:

> What is the approximate linear trend?

So:

```
scatterplot → observations
regplot     → observations + fitted relationship
```

---

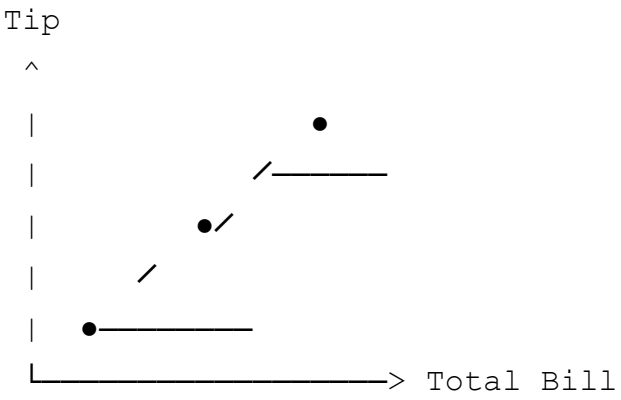
6. Regression with groups — `sns.lmplot()`

If you want regression lines split by a categorical variable, `lmplot()` is often more convenient.

```
sns.lmplot(
    data=df,
    x="total_bill",
    y="tip",
    hue="sex"
)

plt.show()
```

You get something like:



You can also create separate panels:

```
sns.lmplot(
    data=df,
    x="total_bill",
    y="tip",
    col="sex"
)
```

---

7. Pie Chart — Seaborn?

No.

Seaborn does not have a dedicated `pieplot()` function.

Use Matplotlib:

```
plt.pie()
```

For example:

```
counts = df["day"].value_counts()

plt.pie(
    counts,
    labels=counts.index,
    autopct="%1.1f%%"
)

plt.show()
```

So your cheat sheet should remember:

```
Seaborn:
line       → sns.lineplot()
bar        → sns.barplot()
scatter    → sns.scatterplot()
regression → sns.regplot()
histogram  → sns.histplot()
box        → sns.boxplot()

Pie:
           → plt.pie()
```

---

8. Histogram — `sns.histplot()`

What

Shows the **distribution of a numerical variable**.

```
sns.histplot(
    data=df,
    x="total_bill"
)
```

```
plt.show()
```

Conceptually:

```
Frequency
 ^
 |       ▇▇▇
 |     ▇▇▇▇▇▇
 |   ▇▇▇▇▇▇▇▇▇
 | ▇▇▇▇▇▇▇▇▇▇▇
 └──────────────────> Total Bill
```

Control number of bins

```
sns.histplot(
    data=df,
    x="total_bill",
    bins=10
)
```

More bins:

```
bins=20
```

Fewer bins:

```
bins=5
```

The choice of bin width can substantially affect how the distribution looks.

Histogram with KDE

```
sns.histplot(
    data=df,
    x="total_bill",
    kde=True
)
```

You get:

```
Histogram + smooth density curve
```

---

9. Histogram by category

```
sns.histplot(
    data=df,
    x="total_bill",
    hue="sex"
)
```

Or use transparency:

```
sns.histplot(
    data=df,
    x="total_bill",
    hue="sex",
    alpha=0.5
)
```

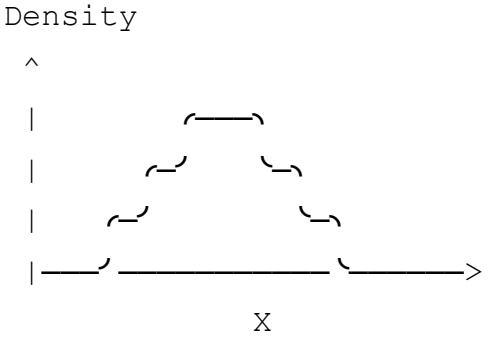
---

10. KDE – related to histogram

Although you didn't specifically ask for KDE, it's worth knowing because you'll see it constantly with Seaborn.

```
sns.kdeplot(
    data=df,
    x="total_bill"
)
```

KDE estimates the underlying probability density.



Histogram:

> actual observations grouped into bins

KDE:

> smooth estimate of the distribution

---

11. Box Plot — `sns.boxplot()`

What

Shows the **distribution of a numerical variable**, especially useful for comparing groups.

```
sns.boxplot(
    data=df,
    x="day",
    y="total_bill"
)

plt.show()
```

Conceptually:

```
Total Bill

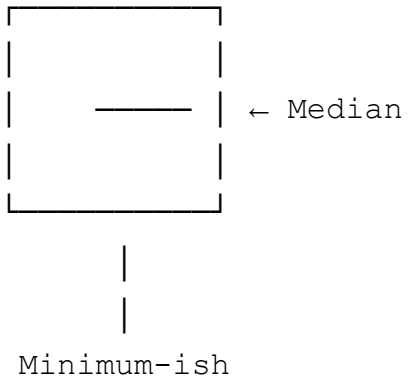
    |
    |        ┌─────┐
    |        |  -  |
    |        └─────┘
    |           |
    |           |
    |           •  ← possible outlier
    |
    └──────────────────

       Fri  Sat  Sun
```

---

12. Understanding a box plot

```
              Maximum-ish
                   |
                   |
```



The box represents:

```
Q1 → 25th percentile
Q2 → Median
Q3 → 75th percentile
```

And:

```
IQR = Q3 - Q1
```

Points beyond the whisker limits are commonly displayed as potential outliers.

---

13. Box plot with hue

```
sns.boxplot(
    data=df,
    x="day",
    y="total_bill",
    hue="sex"
)
```

Now you're comparing:

```
Day
 ↓
Distribution of bills
 ↓
Split by sex
```

This is much more informative than calculating only the mean.

---

14. Horizontal box plot

Swap X/Y:

```
sns.boxplot(
    data=df,
    x="total_bill",
    y="day"
)
```

Very useful when category labels are long.

---

15. The six main Seaborn functions

Here's the core cheat sheet:

```
# LINE
sns.lineplot(
    data=df,
    x="date",
    y="sales"
)

# BAR
sns.barplot(
    data=df,
    x="region",
    y="sales"
)

# SCATTER
sns.scatterplot(
    data=df,
    x="advertising",
    y="sales"
)

# REGRESSION
sns.regplot(
```

```
    data=df,
    x="advertising",
    y="sales"
)

# HISTOGRAM
sns.histplot(
    data=df,
    x="sales"
)

# BOX
sns.boxplot(
    data=df,
    x="region",
    y="sales"
)

# PIE — Matplotlib
plt.pie(
    values,
    labels=labels,
    autopct="%1.1f%%"
)
```

---

16. The most important parameters

Most of your Seaborn charts will repeatedly use these:

```
data=
x=
y=
hue=
style=
size=
col=
row=
palette=
alpha=
```

data

Your DataFrame:

```
data=df
```

x

X-axis column:

```
x="total_bill"
```

y

Y-axis column:

```
y="tip"
```

hue

Color/group:

```
hue="sex"
```

style

Different marker/line styles:

```
style="smoker"
```

size

Map variable to marker size:

```
size="size"
```

col

Create separate columns/panels:

```
col="sex"
```

`row`

Create separate rows/panels:

```
row="time"
```

---

17. `hue` is probably the most important Seaborn feature

Suppose:

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
    hue="sex"
)
```

You have:

```
X = total_bill
Y = tip
Color = sex
```

This lets you add another dimension without manually creating multiple charts.

Similarly:

```
sns.lineplot(
    data=df,
    x="date",
    y="sales",
    hue="region"
)
```

means:

```
X      → Date
Y      → Sales
Color  → Region
```

---

18. palette

Control colors:

```
sns.barplot(
    data=df,
    x="day",
    y="total_bill",
    hue="day",
    palette="Blues",
    legend=False
)
```

Common palettes:

```
"deep"
"muted"
"bright"
"pastel"
"dark"
"colorblind"
"Blues"
"Greens"
"viridis"
```

You can also provide your own:

```
palette = {
    "Male": "#2563eb",
    "Female": "#ec4899"
}
```

---

19. alpha

Controls transparency.

```
sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
```

```
    alpha=0.5
)
```

Useful when many points overlap.

Typical values:

```
0.2 → very transparent
0.5 → semi-transparent
0.8 → mostly opaque
1.0 → opaque
```

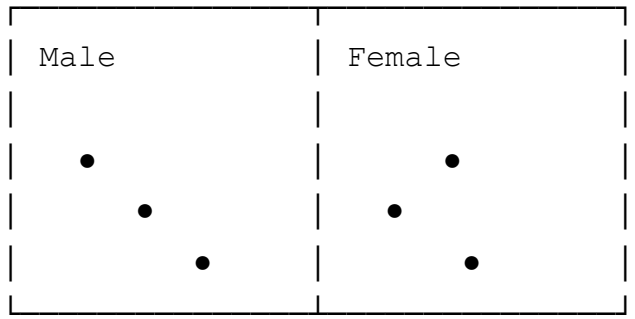
---

20. `col` and `row`: small multiples

This is one of Seaborn's superpowers.

```
sns.relplot(
    data=df,
    x="total_bill",
    y="tip",
    col="sex"
)
```

Instead of one plot:



You can also do:

```
sns.relplot(
    data=df,
    x="total_bill",
    y="tip",
    row="time",
    col="sex"
)
```

This creates a grid.

---

21. Figure-level vs axes-level functions

This is another important Seaborn concept.

Axes-level

Examples:

```
sns.lineplot()
sns.barplot()
sns.scatterplot()
sns.regplot()
sns.histplot()
sns.boxplot()
```

These generally operate on one Matplotlib Axes.

So you can do:

```
fig, ax = plt.subplots()

sns.scatterplot(
    data=df,
    x="total_bill",
    y="tip",
    ax=ax
)
```

This is particularly useful for the **Matplotlib dashboard** you're building.

Figure-level

Examples:

```
sns.relplot()
sns.catplot()
sns.displot()
sns.lmplot()
```

These create/manage their own figure-level layouts.

For example:

```
sns.displot(
    data=df,
    x="total_bill",
    col="sex"
)
```

---

22. A very useful mental model

Seaborn has several families.

Relational

```
sns.scatterplot()
sns.lineplot()

sns.relplot()
```

Question:

> How are variables related?

Categorical

```
sns.barplot()
sns.boxplot()
sns.countplot()
sns.violinplot()
sns.stripplot()
sns.swarmplot()
```

Question:

> How does a numerical variable behave across categories?

Distribution

```
sns.histplot()
sns.kdeplot()

sns.displot()
```

Question:

> What does the distribution look like?

Regression

```
sns.regplot()
sns.lmplot()
```

Question:

> What is the relationship/trend between numerical variables?

---

23. Quick chart-selection cheat sheet

```
I have a DATE/TIME + number
        ↓
    lineplot()

I have CATEGORY + number
        ↓
    barplot()

I have NUMBER + NUMBER
        ↓
    scatterplot()

I have NUMBER + NUMBER
and want a fitted trend
        ↓
    regplot()

I have ONE numerical variable
and want its distribution
        ↓
```

```
    histplot()

I have CATEGORY + numerical distribution
and want to compare groups
        ↓
    boxplot()

I need PART-TO-WHOLE
        ↓
    plt.pie()
```

---

24. Cheat sheet with `tips` dataset

Here's a compact practice set you can run directly:

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset("tips")

# 1. LINE
sns.lineplot(
    data=df,
    x="size",
    y="tip"
)
plt.show()

# 2. BAR
sns.barplot(
    data=df,
    x="day",
    y="total_bill"
)
plt.show()

# 3. SCATTER
sns.scatterplot(
    data=df,
    x="total_bill",
```

```
    y="tip",
    hue="sex"
)
plt.show()

# 4. REGRESSION
sns.regplot(
    data=df,
    x="total_bill",
    y="tip"
)
plt.show()

# 5. HISTOGRAM
sns.histplot(
    data=df,
    x="total_bill",
    bins=15,
    kde=True
)
plt.show()

# 6. BOX PLOT
sns.boxplot(
    data=df,
    x="day",
    y="total_bill"
)
plt.show()

# 7. PIE
counts = df["day"].value_counts()

plt.pie(
    counts,
    labels=counts.index,
    autopct="%1.1f%%"
)
plt.show()
```

One caveat with the first example: `size` is ordered/discrete rather than time, so a line chart there is mainly useful for demonstrating the API; in a real analysis, you'd generally use a line chart when the

ordering itself has analytical meaning.

⭐ The "don't forget this" version

```
# Trend
sns.lineplot(data=df, x="date", y="sales")

# Category comparison
sns.barplot(data=df, x="category", y="sales")

# Two numerical variables
sns.scatterplot(data=df, x="x", y="y")

# Numerical relationship + regression
sns.regplot(data=df, x="x", y="y")

# Distribution
sns.histplot(data=df, x="value")

# Distribution across groups
sns.boxplot(data=df, x="group", y="value")

# Part-to-whole
plt.pie(values, labels=labels)
```

And the **Seaborn superpower** to remember is:

```
hue=
```

For example:

```
sns.scatterplot(
    data=df,
    x="Sales",
    y="Profit",
    hue="Region",
    style="Product",
    size="Customers",
    alpha=0.7
)
```